

Lab-2 Worksheet

Jahanvi Gajera (M25CSA012)

February 1, 2026

1 Introduction

The objective of this experiment is to design, train, and analyze a Convolutional Neural Network (CNN) using the CIFAR-10 dataset. The experiment focuses on model training, computational complexity analysis, gradient and weight flow visualization, and performance evaluation using Weights & Biases (WandB).

2 Dataset Description

The CIFAR-10 dataset consists of 60,000 RGB images of size 32×32 , distributed across 10 classes. The dataset is split into 50,000 training images and 10,000 validation images. Data normalization and augmentation techniques were applied to improve learning stability and generalization.

3 Custom DataLoader

A custom PyTorch Dataset class was implemented to load CIFAR-10 images. The DataLoader applies the following transformations:

- Padding & Cropping
- Conversion to tensor
- Normalization with mean and standard deviation of dataset
- Apply ColourJitter
- Random Erasing: Advanced technique for robustness

Batch loading with shuffling was enabled for training data to improve convergence.

4 CNN Model Architecture

The CNN architecture consists of:

- Three convolutional layers with 32, 64, and 128 filters
- ReLU activation functions

- MaxPooling layers for spatial downsampling
- Fully connected layers for classification

This architecture provides a balance between representational capacity and computational efficiency.

5 FLOPs and Parameter Analysis

The computational complexity of the model was estimated using the `ptflops` library. Both FLOPs and total trainable parameters were logged to WandB. This analysis helps in understanding the efficiency of the model relative to its achieved accuracy.

6 Training Configuration

The model was trained using the following setup:

- Optimizer: Adam
- Learning Rate: 0.001
- Loss Function: Cross-Entropy Loss
- Batch Size: 128
- Number of Epochs: 50

7 Gradient Flow and Weight Update Visualization

Gradient flow was monitored by logging the gradient norms of all trainable layers during backpropagation. Weight update distributions were visualized using histograms after training. These visualizations were logged and analyzed using WandB to detect issues such as vanishing or exploding gradients.

8 Results

During training, the model achieved a steady decrease in training loss and a consistent improvement in training accuracy. By epoch 50, training accuracy reached approximately 83.5%, while training loss reduced to around 0.47.

However, validation accuracy remained significantly lower, fluctuating between 59% and 65%. Validation loss remained high and unstable, with noticeable spikes across multiple epochs.

9 Observations and Analysis

The following observations were made from the experiment:

- Training accuracy consistently improved, showing that the CNN successfully learned discriminative features from the training set.
- Validation accuracy plateaued after approximately 30 epochs and showed strong fluctuations, indicating poor generalization.
- Validation loss remained high and exhibited sharp spikes, suggesting model sensitivity to unseen data.
- A growing gap between training and validation accuracy was observed after epoch 30, which is a classic indicator of overfitting.
- Gradient flow visualizations showed stable gradients in early layers, while deeper layers exhibited smaller gradient magnitudes.
- Weight distribution plots revealed increasing concentration of weights over epochs, suggesting memorization of training patterns.

10 Overfitting Discussion

From epochs 32 onward, the model exhibited clear signs of overfitting. While training performance continued to improve, validation performance stagnated and degraded. This indicates that the model was learning training-specific representations rather than generalizable features.

11 WandB Reporting

All metrics including training loss, validation loss, accuracy, gradient flow, weight distributions, and FLOPs were logged to Weights & Biases. WandB provided interactive dashboards that enabled effective monitoring and analysis of model behavior throughout training.

12 Links

- Github Link: https://github.com/jahanvi0106/MLOps-Jahanvi_Gajera-M25CSA012/tree/Lab_2_Worksheet
- wandb link: https://wandb.ai/m25csa012-iit-jodhpur/Jahanvi_Gajera_M25CSA012_lab2_worksheet/workspace?nw=nwuserm25csa012